PRESIDENCE DU CONSEIL

MINISTERE DE LA PLANNIFICATION DU DEVELOPPEMENT ET DE LA COOPERATION

Institut Africain d'Informatique
Etablissement Inter-Etats d'Enseignement Supérieur
Représentation du Togo (IAI-TOGO)
07 **BP** 12456 Lomé 07 **Tél.** : (+228) 22.20.47.00
E-mail : iaitogo@iai-togo.tg **Site Web**: www.iai-togo.tg

Support de cours

Conception et Implémentation des Structures de Données (Python)

Chargé de cours : M. **AKOMAGBE** Innocent

Dernière mise à jour: 01/04/2026

```
56 68 78 6F 76 20 66 68 78 61 20 74 78 6C 20 76 68 20 75 6C 76 74 78 68 75
72 71 77 20 64 20 73 68 78 77 2D 68 77 75 68 20 64 6F 6F 68 75 20 77 75 72
73 20 6F 72 6C 71 20 76 64 78 75 72 71 77 20 6D 78 76 74 78 27 72 78 20 6C
6F 20 68 76 77 20 73 72 76 76 6C 65 6F 68 20 67 27 64 6F 6F 68 75 2E
```

Année Académique 2024 - 2025

TABLE DES MATIERES

INTRODUCTION 4
PARTIE I: INTRODUCTION A PYTHON 5
Chapitre 1 : Prise en main de python 6
I. Présentation de python 6
II. Carte d'identité de Python 6
III. Quelques caractéristiques de Python 6
IV. Installation des outils nécessaires 7
V. L'interpréteur de python 8
VI. Les unités lexicales 9
VII. Premier programme 10
Chapitre 2 : Les variables 13
I. Affectation 13
II. Réaffectation 13
III. Affectations multiples 14
IV. Affectations parallèles 14
V. Règles de nommage d'une variable 14
VI. Les types de variables 15
VII. Écriture scientifique 15
VIII. Simplification de l'écriture de gros nombre 16
IX. Conversion 16
Chapitre 3 : Les entrées et sorties 17
I. Les entrées 17
II. Les sorties 17
Chapitre 4 : Les opérateurs 24
I. Quelques définitions 24
II. Les opérateurs de Python 24
Chapitre 5 : Les instructions conditionnelles 36
I. L'instruction if 36
II. L'instruction if ... else 36
III. L'instruction if ... elif ... else 37
IV. L'instruction match 38
Chapitre 6 : Les instructions répétitives 41
I. La boucle while 41
II. La boucle for 42

III. Instructions de contrôle de boucle 43
Chapitre 7 : Les chaines de caractères 44
I. Création des chaines de caractères 44
II. Accès aux caractères individuels d'une chaîne 45
III. Suppression / Mise à jour d'une chaîne 46
IV. Formatage des chaînes 46
V. Parcours d'une chaine de caractères 47
VI. Accès au code ASCII 48
VII. Quelques fonctions de manipulation des chaines 48
Chapitre 8 : Les fonctions 53
I. Généralité 53
II. La déclaration d'une fonction 53
III. L'appel d'une fonction 53
IV. Notions de variables locales et globales 54
V. Un peu plus sur la définition des fonctions 55
VI. Arguments de longueur variable 56
VII. Fonctions anonymes 57
VIII. Fonctions récursives 58
IX. Chaînes de documentation 58
X. Les modules 59
Chapitre 9 : Les fichiers 61
I. Ouverture d'un fichier 61
II. Traitement sur un fichier 62
III. Fermeture d'un fichier 65
IV. L'instruction with 65
Chapitre 10 : Les exceptions 66
I. Généralités 66
II. La gestion des erreurs 66
III. Lever d'une exception 68
IV. Contrôles de saisies 68
V. Les assertions 68
PARTIE II: LES STRUCTURES DE DONNEES EN PYTHON 70
Chapitre 1 : Présentation et intérêt des structures de données 71
I. Quelques rôles essentiels des structures de données : 71
II. Implémentation des structures de données 71
Chapitre 2 : Les listes 73

I. Généralité 73
II. Déclaration d'une liste 73
III. Accès aux éléments d'une liste 73
IV. Opérations sur les listes 74
V. Liste de liste 80
VI. Les tableaux de type array 80
VII. Les listes en compréhension (List Comprehension) 81
Chapitre 3 : Les ensembles 83
I. Généralité 83
II. Création d'ensemble 83
III. Manipulation des éléments d'un set 83
IV. Opération sur les ensembles 85
V. Les frozensets 86
Chapitre 4 : Les tuples 87
I. Définition 87
II. Création d'un tuple 87
III. Opérations sur les tuples 87
IV. Application des tuples 90
Chapitre 5 : Les dictionaries 92
I. Définition 92
II. Création d'un dictionnaire 92
III. Opérations sur les dictionnaires 93
IV. Méthodes de dictionnaire 94
V. Liste de dictionnaires 96
VI. Les tris sur les dictionnaires 96
Chapitre 6 : Résumé des structures de données de type container 98
Chapitre 7 : Annexe 99
I. Quelques fonctions supplémentaires sur les chaines 99
II. Quelques méthodes disponibles avec l'objet file 101

INTRODUCTION

Le développement de système d'informations implique la manipulation de données devant être organisées de manière optimisé afin de faciliter les opérations comme la recherche, l'ajout de nouvelles informations ou la suppression. Les structures de données nous permettent d'atteindre cet objectif

L'étude des structures de données revêt une importance cruciale dans le domaine de l'informatique et du développement logiciel. Leur principal objectif est d'organiser et de stocker des données de manière efficace afin de faciliter leur manipulation, leur recherche et leur accès.

Ce cours enseignera dans un premier temps les fondamentaux du langage de programmation Python et ensuite l'implémentation des structures de données avec Python.

PARTIE I: INTRODUCTION A PYTHON

I. Présentation de python

Le langage python est un langage de programmation de haut niveau, interprété et facile à apprendre. Il dispose de structures de données de haut niveau et permet une approche simple mais efficace de la programmation orientée objet.

Grâce à sa syntaxe élégante et son typage dynamique, Python est un langage idéal pour l'écriture de scripts et le développement rapide d'application.



Figure 1: Logo de python

II. Carte d'identité de Python

Python est un langage de programmation interprété, multiparadigme et multiplateformes. Il favorise la programmation impérative structurée, fonctionnelle et orientée objet. Il a été créé par Guido van Rossum en 1989). Sa dernière version est le `3.13.2` publié le 4 février 2025.

- **`Python 1.0`** est officiellement publié en janvier 1994 et ajoute des outils de programmation fonctionnelle tels que `lambda`, `map`, `filter` et `reduce`
- **`Python 2.0`** est publié en octobre 2000
- **`Python 3.0`** est publié en décembre 2008
- **`Python 3.10`** est publié en octobre 2021

III. Quelques caractéristiques de Python

Python est un langage de programmation polyvalent et populaire, apprécié pour sa simplicité, sa lisibilité et sa puissance.

- **Une lisibilité élevée :** La syntaxe simple, claire et intuitive de python favorise la lisibilité du code. Les blocs de code sont délimités par l'indentation, ce qui le code facile à comprendre pour les débutants.
- **Un langage interprété :** Python est un langage interprété, ce qui signifie qu'il n'a pas besoin d'être compilé avant d'être exécuté. Cela permet un développement plus rapide et un débogage plus facile.

- **Une bibliothèque standard riche :** Python dispose d'une vaste bibliothèque standard qui offre des modules et des packages pour effectuer une variété de tâches, telles que la manipulation de fichiers, le traitement des chaînes, la gestion des entrées/sorties, etc. Cela permet aux développeurs d'écrire des programmes complexes sans avoir à réinventer la roue.
- **Portabilité :** Python est un langage multiplateforme, c'est-à-dire qu'il peut être exécuté sur différentes plates-formes telles que Windows, MacOs et Linux sans nécessiter de modifications importantes du code source.
- **Dynamique et typage fort :** Python est un langage dynamiquement typé, ce qui signifie que les types des variables sont déterminés à l'exécution plutôt qu'à la compilation. Cependant, il possède un typage fort, ce qui signifie que les types des variables sont strictement contrôlés.
- **Grande communauté et écosystème :** Python bénéficie d'une vaste communauté de développeurs actifs qui contribuent à son écosystème en créant des bibliothèques, des frameworks et des outils qui étendent les capacités du langage.

IV. Installation des outils nécessaires

Avant de nous mettre à la programmation avec python, nous devons installer l'environnement avec les outils nécessaires.

1. Installation de l'environnement

Les fichiers d'installation de python sont disponibles sur le site web officiel de python : http://www.python.org/download.. Python étant un langage multiplateforme, il est possible de l'installer sur la plupart des systèmes d'exploitation.

2. Installation d'un éditeur de texte

Même s'il est possible d'utiliser l'interpréteur de python de façon interactive, il est pénible d'écrire tout un programme avec ce mode interactif. Dans le cadre de ce cours, nous utiliserons un éditeur de code pour l'édition de nos programmes.

Comme éditeur de code nous disposons de `VsCode`, `Eclipse`, `Pycharm`, `Notepad++`, etc. Toutefois, nous choisirons `VsCode` pour sa simplicité, sa popularité et les fonctionnalités qu'il offre.

V. L'interpréteur de python

1. Le mode interactif de python

Python dispose d'un interpréteur en mode interactif qui nous permet d'écrire des instructions qui sont directement exécutées par l'interpréteur. Une fois l'instruction exécutée, les résultats sont affichés dans la console.

2. Lancement de l'interpréteur

Pour lancer l'interpréteur interactif de python, il suffit de saisir *python3* ou *python* dans ligne de commande sur MacOs et Linux, ou python sous Windows. Notons que Python dispose également d'un éditeur intégré nommé IDLE que nous pouvons également lancer.

Pour lancer IDLE, il suffit de rechercher et d'ouvrir l'application "IDLE Python" sous Windows.

Les caractères chevrons « >>> » également appelés *prompt* constituent le signal d'invite, ou

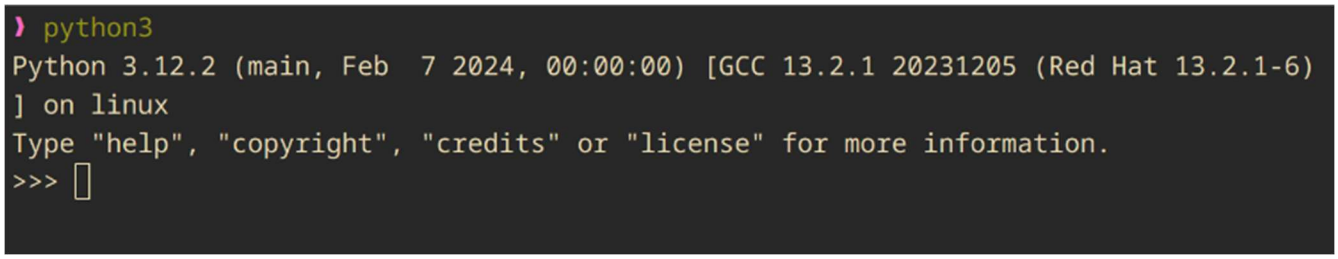


Figure 2: Exemple de lancement d'un interpréteur de Python

prompt principal, qui vous indique que Python est prêt à exécuter une commande.

En mode interactif, les commandes tapées sont immédiatement exécutées et le résultat est affiché. Mais le plus souvent les programmeurs préfèrent écrire le code de leur script Python contenant plusieurs instructions dans un fichier avec l'extension .py et sauvegardé sur un disque dur.

3. Python comme une calculatrice

L'interpréteur de Python peut être utilisé pour tester l'exécution de quelques lignes de code. Il peut être également utilisé comme une simple calculatrice. Les opérateurs +, -, * et / peuvent être utilisés pour effectuer des opérations arithmétiques. Les parenthèses (()) peuvent être utilisées pour grouper les opérations.

- **Quelques exemples de commandes**

```
>>>7+4
11

>>>7+3*4
19
#la hiérarchie des opérations mathématiques est bien respectée

>>>(7+3)*4
40
>>>20/33
0.6060606060606061

>>>4*2.5/3.3
3.0303030303030303
>>>5/2
2.5
# La division en Python retourne toujours comme résultat un réel
```

VI. Les unités lexicales

1. Les mots réservés

Python dispose de 33 mots réservés cités ci-dessous :

and	for	return	del	lambda
exec	print	continue	in	True
or	break	if	with	False
as	from	try	elif	except
finally	raise	def	is	not
pass	class	import	yield	
assert	global	while	else	

2. Les commentaires

Comme tous les langages de programmation modernes, Python autorise les commentaires dans un programme. Ils servent à la documentation du programme et sont destinés aux lecteurs humains. Ils n'ont aucune incidence sur le code et sont totalement ignorés lors de la compilation. Ils peuvent être placés à tout endroit d'un programme ou l'espace est autorisé et peuvent être sur une seule ligne comme sur plusieurs lignes.

Les commentaires en Python commencent avec un caractère croisillon, #, et s'étendent jusqu'à la fin de la ligne. Un commentaire peut apparaître au début d'une ligne ou à la suite d'un espace ou de code, mais pas à l'intérieur d'une chaîne de caractères littérale.

Exemple :

```
1. # une ligne de commentaire
2. print('Bonjour le monde') # Une autre ligne de commentaire
3. texte = "#  Ceci n'est pas un commentaire"
```

VII. Premier programme

Notre premier programme aura pour objectif d'afficher « **`Bonjour le monde`** » à l'écran.

- **Avec l'aide de l'interpréteur**

Dans votre interpréteur, tapez, le code suivant `print('Bonjour le monde')`.

```
1. >>>print('Bonjour le monde')
2. Bonjour le monde
```

On remarque que l'instruction est exécutée par l'interpréteur et la chaine de caractères « `Bonjour le monde` » est affichée.

- **Avec l'aide de l'éditeur de code VS Code**

Sur votre disque, dans un dossier Python, créer un fichier nomme `bonjour.py`. Ensuite, ouvrer ce fichier dans `VSCode` et réécrivez le code précèdent à l'intérieur du fichier.

Nous pouvons exécuter le script avec la commande suivante :

```
$ python bonjour.py
```

1. Structure d'un programme

a. Les instructions

Un programme Python est composé d'instructions. Une instruction est un ordre unitaire donné à un programme. Par exemple afficher « `Bonjour le monde` » est une instruction, de même qu'effectuer une opération.

La structure d'un programme Python est assez différente de celle des langages plus traditionnels comme le C.

En Python, les limites des instructions et des blocs sont définies par l'indentation[1].

[1] Indentation : Selon Wikipédia, l'indentation consiste en l'ajout de tabulations ou d'espaces dans un fichier, pour une meilleure lecture et compréhension du code. Le style d'indentation décrit les différentes manières que les programmeurs utilisent pour faire ressortir un bloc de code.

- **Le point-virgule (;)** à la fin de chaque instruction est remplacé par le caractère de fin de ligne (symbolisé par `"\n"`). Ce qui simplifie la syntaxe, car traditionnellement les points-virgules étaient toujours suivis de retours à la ligne.
- **Les accolades ({})** fréquemment utilisées pour définir les blocs sont remplacés par les niveaux d'indentation. En effet, il était déjà conseillé aux programmeurs de se servir aussi des sauts à la ligne et de l'indentation pour bien délimiter visuellement les blocs, donc c'est également un allègement du code.

En définitive, Python oblige à écrire du code lisible, et à prendre de bonnes habitudes conservables dans d'autres langages, ce qui en fait un langage idéal pour apprendre la programmation.

b. Les espaces sont ignorés.

À part ceux qui servent à l'indentation, en début de ligne, les espaces placés à l'intérieur des instructions sont ignorés, sauf s'ils font partie d'une chaîne de caractères.

L'interpréteur Python commence par analyser la première ligne :

- Si celle-ci contient une instruction, alors il l'exécute ;
- Si l'instruction n'est pas une instruction de contrôle, alors, il passe à la ligne suivante, l'analyse et l'exécute ;
- Si le programme Python arrive à la fin du fichier à exécuter, alors, il sort du programme et en arrête l'exécution.

2. Notion de bloc d'instructions et d'indentation

Un bloc d'instructions est une suite d'instructions qui est alignée sur la même tabulation. Les blocs d'instructions sont créés par les instructions de contrôles comme `if`, `while` et `for`, ainsi que par les instructions permettant de déclarer des fonctions.

Sous Python, toutes les instructions composées ont toujours la même structure : une ligne d'en-tête terminée par un double point, suivie d'une ou de plusieurs instructions indentées sous cette ligne d'en-tête.

Nous disposons de deux façons pour indenter en Python : La première façon consiste à utiliser quatre espaces contrairement à la deuxième façon qui utilise un seul caractère tabulation. Cependant, il est interdit de mélanger les deux sous peine d'erreurs de type *`IndentationError: unindent does not match any outer indentation level`*.

La tabulation est notamment la convention utilisée pour la librairie standard de Python, mais les espaces ont l'avantage de fonctionner avec tous les éditeurs.

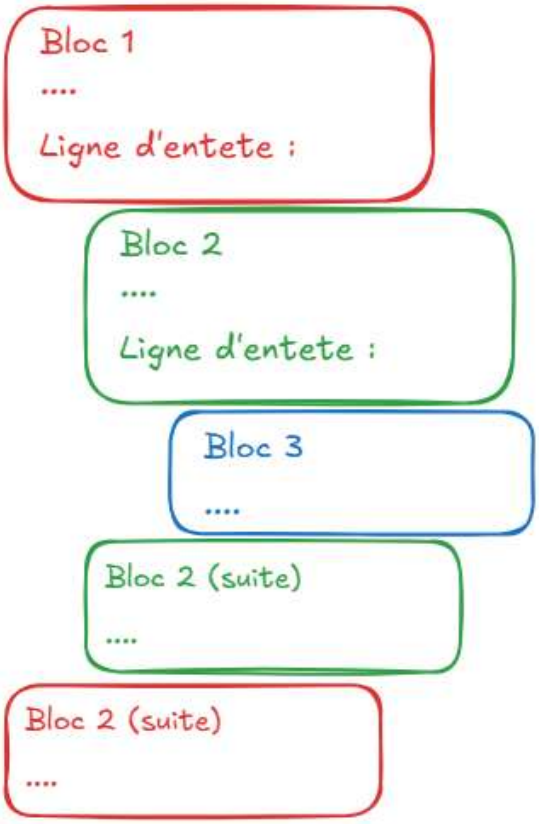


Figure 3: Schéma montrant les niveaux d'indentation en Python

Les résultats prévenants des calculs que nous effectuons dans l'interpréteur sont perdues après affichage. Pourtant, il pourrait être utile de conserver un résultat, afin de le réutiliser par la suite dans d'autres calculs.

Le concept des variables en Python est similaire à ceux des autres langages de programmation tels que C, Pascal, C++, Java. L'objectif est d'associer un nom à une valeur, dans le but de stocker cette valeur en mémoire RAM et pouvoir la retrouver (grâce à son nom).

Python ne dispose d'une instruction spéciale pour la déclaration des variables. La déclaration d'une variable se fait lors de son initialisation ou sa première affectation.

I. Affectation

En Python comme dans de nombreux autres langages, le symbole `'='` est l'opérateur d'affectation. Cette valeur restera accessible jusqu'à la fin de l'exécution du programme.

La valeur de la variable peut être un littéral, c'est-à-dire une constante, ou bien une expression (ex : une autre variable ou une instruction comme `''1+1''`). L'expression est évaluée avant d'être affectée à la variable.

Exemple :

```
jour = 'lundi'
```

NB : En Python, les constantes sont déclarées et affectées comme les variables. La convention **PEP8** leur impose juste un nom en majuscules

II. Réaffectation

Si la variable affectée n'existe pas, l'ordinateur la crée, sinon il écrase sa valeur. En effet, il est permis de réaffecter une nouvelle valeur à une même variable, autant de fois qu'on le souhaite.

```
1. >>> jour = 'lundi'
2. >>> print(jour)
3. lundi
4. >>> jour = 'vendredi'
5. >>> print(jour)
6. vendredi
```

III. Affectations multiples

Sous Python, on peut assigner une valeur à plusieurs variables simultanément.

Exemple d'affectation multiple :

```
1. >>> x = y = 7
2. >>> x
3. 7
4. >>> y
5. 7
```

IV. Affectations parallèles

On peut aussi effectuer des affectations parallèles à l'aide d'un seul opérateur :

```
1. >>> a, b = 4, 8.33
2. >>> a
3. 4
4. >>> b
5. 8.33
```

V. Règles de nommage d'une variable

Sous Python, les noms de variables doivent en outre obéir à quelques règles simples :

- Un nom de variable est une séquence de lettres (a → z, A → Z) et de chiffres (0 → 9), qui doit toujours commencer par une lettre.
- Python est sensible à la casse (les caractères majuscules et minuscules sont distingués). Donc `"Joseph"`, `"joseph"`, et `"JOSEPH"` sont des variables différentes.
- Les 29 « mots réservés » du langage sont déjà pris (ex : `"print"`).
- Seules les lettres du code ASCII sont autorisées. Les lettres accentuées, les cédilles, les espaces, les caractères spéciaux tels que $, #, @, etc. sont interdits, à l'exception du caractère _ (souligné).

À partir de Python 3, les règles de nommage ont été assouplies, car il devient possible d'utiliser n'importe quelle lettre Unicode non-opérateur dans les noms. Par exemple, français = 1 est autorisé, mais pas f_en_€ = 6.55957.

Il faut également noter que les variables dont le nom commence par le caractère _ ont une signification particulière :

- Les noms commençant par un _ ne sont pas exportés lorsqu'ils se trouvent dans un module ;
- Les noms commençant par deux _ et finissant par deux _ sont réservés par le langage lui-même, notamment pour la programmation orientée objet.

VI. Les types de variables

Le type d'une variable correspond à la nature de celle-ci. Elle détermine les valeurs possibles qu'une variable peut posséder ainsi que les opérations qu'il est possible d'effectuer.

- Pour représenter les types numériques, nous avons :
  - **Le type entier (`int`)** permet de représenter n'importe quel nombre entier, peu importe sa taille.
  - **Le type flottant (`float`)** permet de représenter des nombres comportant une partie décimale, compris entre 10^{-308} et 10^{308}. La valeur spéciale `math.inf` représente l'infini.
  - **Le type complexe (`complex`)** permet de représenter des nombres complexes, où le nombre imaginaire se note `j`.
- Pour les chaînes de caractères, nous avons le type **`str`**

En dehors des types cités précédemment, Python dispose d'autres types comme les booléens, les sets, les dictionnaires, les listes, les tuples, etc.

VII. Écriture scientifique

On peut écrire des nombres très grands ou très petits avec des puissances de `10` en utilisant le symbole `e`.

```
1. >>>1e9
2. 1000000000.0
3. >>>3.14e-3
4. 0.00314
```

Exemple :

On appelle cela écriture ou notation scientifique. On pourra noter deux choses importantes :

- `1e6` ou `3.12e-3` n'implique pas l'utilisation du nombre exponentiel `e`, mais signifie 1×10^{6} ou 3.12×10^{-3} respectivement ;
- Même si on ne met que des entiers à gauche et à droite du symbole `e` (comme dans `1e6`), Python génère systématiquement un `float`.

VIII. Simplification de l'écriture de gros nombre

Il est parfois pénible de s'y retrouver quand on manipule de grand nombre ou un nombre avec plusieurs zéros. Afin de résoudre ce problème, Python autorise l'utilisation du caractère « souligné » (ou underscore) _ pour séparer des groupes de chiffres.

Exemple :

```
>>>salaire=1_000_000
>>>salaire
1000000
>>>nombre_avogadro=6.022_140_76e23
>>>nombre_avogadro
6.02214076e+23
```

IX. Conversion

Il est possible de convertir explicitement une donnée d'un type vers un autre, en utilisant des fonctions prédéfinies. Les fonctions `int`, `float`, `complex` et `str` permettent de convertir une donnée vers les types correspondants.

```
>>> a = 14
>>> print(type(a))
<class 'int'>
>>> b = 3.14
>>> print(type(b))
<class 'float'>
>>> a = str(a)
>>> print(a)
14
>>> b = int(b)
>>> print(b)
3
```

Les entrées et les sorties en Python sont fondamentales pour la communication entre l'utilisateur et le programme.

I. Les entrées

Les entrées (entrée) permettent à l'utilisateur de transmettre des données au programme, souvent à travers des fonctions telles que la fonction **`input`**, qui recueille les données sous forme d'une chaîne de caractères.

Pour récupérer la saisie de l'utilisateur, nous disposons de la fonction **`input`**. Elle renvoie une chaine de caractères lue depuis l'entrée standard. Nous pouvons ensuite utiliser les fonctions de conversions pour convertir cette chaine de caractères vers le type désiré.

```
>>>nom = input('Quel ton nom? ')
Quel ton nom? root
>>> print(nom)
root

>>> age = int(input('Quel est ton age? '))
Quel est ton age? 11
>>> print(type(age))
<class 'int'>
>>> print(age)
11
```

II. Les sorties

L'affichage à travers le périphérique de sortie standard se fait à partir de la fonction `print`. La fonction `print()` affiche l'argument qu'on lui passe entre parenthèses et un retour à ligne. Ce retour à ligne supplémentaire est ajouté par défaut. Si toutefois, on ne veut pas afficher ce retour à la ligne, on peut utiliser l'argument par « mot-clé » `end` :